

SRE/DevOps Interview Guide: SLOs, SLIs, and Error Budgets

This guide provides a structured response to the question: "Tell us about how you've defined SLOs/SLIs for a critical service. How did you set thresholds, manage error budgets, and make tradeoffs between feature velocity and reliability? Provide an example where you enforced error budget policies."

Introduction

"In my experience, defining and managing Service Level Objectives (SLOs) and Service Level Indicators (SLIs) is fundamental to Site Reliability Engineering (SRE) and DevOps practices. It allows us to objectively measure service health, align business goals with operational realities, and make data-driven decisions regarding reliability versus feature development. I've applied these principles to critical services, including a Global Payment Processing Engine and, more recently, in the context of Fast-Moving Consumer Goods (FMCG) operations."

1. Defining SLOs/SLIs for a Critical Service (e.g., Global Payment Processing Engine)

"For a critical service like a Global Payment Processing Engine, we start by identifying **Service Level Indicators (SLIs)** that directly reflect user experience and business impact. We typically leverage the **Four Golden Signals**:

- **Availability:** Defined as the proportion of successful payment requests, excluding user-side errors. Measured as $(\text{Successful HTTP 200/201}) / (\text{Total Valid Requests})$.
- **Latency:** The time taken for a transaction to complete. We focus on **P95 and P99 response times** at the API Gateway.
- **Error Rate:** The percentage of requests failing due to internal system errors. Measured as $(\text{HTTP 5xx Errors}) / (\text{Total Requests})$.
- **Throughput:** The volume of transactions processed per second.

Once SLIs are defined, we set **Service Level Objectives (SLOs)**, which are the target values for these SLIs. This is a collaborative process involving engineering, product, and business stakeholders:

- **Historical Baseline:** Analyze past performance data to understand the system's natural behavior.

- **User Expectation Mapping:** For instance, user research might indicate that payments exceeding 2 seconds are perceived as slow. This would inform a **P99 Latency SLO of < 1,500ms**.
- **Business Impact Analysis:** Quantify the financial or reputational cost of unreliability. A 0.1% drop in availability for a payment engine could mean significant revenue loss, justifying a **99.9% Availability SLO** over a 28-day rolling window."

2. Managing Error Budgets

"The **Error Budget** is the inverse of the SLO – it's the maximum allowable unreliability for a service within a given period. For a 99.9% Availability SLO, the error budget is 0.1%.

- **Calculation:** If a service handles 10 million transactions monthly with a 99.9% availability SLO, the error budget allows for 10,000 failed transactions.
- **Monitoring:** We continuously monitor the **Burn Rate**, which indicates how quickly the error budget is being consumed. A burn rate of 1.0 means the budget is being consumed exactly as planned, while a rate of 10.0 signifies a critical issue.
- **Tradeoffs (Feature Velocity vs. Reliability):** The error budget acts as a crucial mechanism for balancing innovation with stability. When the budget is healthy, teams can prioritize new features and aggressive deployments. However, when the budget is depleted, the focus shifts entirely to reliability work, technical debt reduction, and hardening the system. This ensures that reliability is not an afterthought but a measurable constraint on development velocity."

3. Real-Time Implementation in FMCG: An Example of Error Budget Enforcement

"To illustrate error budget enforcement, let's consider an FMCG company's e-commerce and supply chain operations, where real-time data is paramount. Here, reliability extends beyond traditional IT metrics to encompass physical inventory accuracy and order fulfillment speed.

FMCG-Specific SLIs:

- **E-commerce: Add-to-Cart Success Rate** (percentage of successful stock reservations).
- **Inventory: Stock Synchronization Delta** (difference between physical and digital stock levels).
- **Logistics: Last-Mile Tracking Freshness** (age of delivery vehicle GPS coordinates).

Real-Time Architecture (Prometheus & Grafana Focus):

We implement a robust, pull-based observability architecture centered around Prometheus and Grafana, even without OpenTelemetry. This involves:

1. **Instrumentation & Metric Exposure:** Our existing ERP, WMS, and e-commerce microservices are instrumented to expose metrics in a Prometheus-compatible format. This is typically achieved through:
 - **Application-level Exporters:** Libraries (e.g., Prometheus client libraries for Java, Python, Go) embedded directly within the application code to expose custom business metrics (e.g., `add_to_cart_success_total`, `inventory_sync_delta_seconds`).
 - **Generic Exporters:** Using existing Prometheus exporters (e.g., `node_exporter` for host metrics, `jmx_exporter` for JVM applications, `blackbox_exporter` for external endpoint checks) to gather system-level and service-level metrics.
 - **Custom Adapters/Scrapers:** For legacy systems or third-party APIs, we develop lightweight services that poll data and expose it as Prometheus metrics.
2. **Metric Collection & Storage:** Prometheus servers are configured to scrape these exposed metrics endpoints at regular intervals (e.g., every 15-30 seconds). Prometheus then stores this time-series data efficiently.
3. **Real-Time Aggregation & Alerting:** Prometheus uses its powerful query language (PromQL) to calculate SLIs (e.g., availability, latency percentiles, error rates) over short sliding windows. Prometheus Alertmanager is then configured to trigger alerts based on these SLI calculations, especially when error budget burn rates exceed predefined thresholds.
4. **Visualization & Dashboards:** Grafana is used to create dynamic dashboards that visualize these SLIs and, critically, the **Error Budget Burn Rate**. This provides real-time visibility into service health and allows teams to quickly identify when the error budget is being consumed too rapidly.

Error Budget Enforcement Example: The 'Ghost Inventory' Incident

Imagine a scenario where a synchronization lag between the warehouse and the e-commerce platform leads to 'Ghost Inventory' – customers ordering items that are physically out of stock. This directly impacts customer satisfaction and incurs operational costs.

1. **Detection:** The SLI for 'Inventory Sync Delta' (e.g., difference between physical and digital stock) exceeds its threshold, indicating a significant discrepancy.
2. **Budget Impact:** The error budget for 'Order Integrity' starts burning at an accelerated rate (e.g., 20x the normal rate).

3. **Automated Enforcement Actions:** Our policy triggers a multi-level response:

- **Level 1 (Yellow - Warning):** If 50% of the budget is consumed, the system automatically adjusts the displayed 'Safety Stock' on the website (e.g., if 10 items are physically available, only 5 are shown to prevent overselling).
- **Level 2 (Red - Critical):** If 80% of the budget is consumed, the 'Express Delivery' option is temporarily disabled to allow more time for warehouse stock reconciliation.
- **Level 3 (Enforcement - Feature Freeze):** If the budget is fully depleted, a **Feature Freeze** is automatically imposed on the Warehouse Management System (WMS) API. This prevents any new updates from further destabilizing the inventory synchronization, forcing the team to prioritize fixing the underlying issue.

Outcome:

This enforcement ensures that the engineering and operations teams immediately shift focus to resolving the 'Ghost Inventory' problem. By halting new feature deployments or changes to the WMS, we prevent further customer dissatisfaction and financial losses, ultimately restoring the integrity of the inventory system and protecting the brand's reputation."

Conclusion

"In summary, by meticulously defining SLIs and SLOs, establishing clear error budgets, and implementing automated enforcement policies, we create a robust framework that not only measures reliability but actively governs development and operational priorities. This allows organizations, even in dynamic environments like FMCG, to strategically balance the need for rapid innovation with the imperative of maintaining highly reliable and performant services."